AssignmentTwo

Arrays & Procedural routines

By: Eng. Islam Hamdy Mohamed

Guided By: Eng. Hassan Khaled

1. Snippets for the implemented code

```
// Using 'logic' instead of 'reg' or 'wire' because it is a 4-valued logic data type
// (0, 1, X, Z) that permits both procedural assignments and continuous assignments,
// preventing any net vs. variable type conflicts in SystemVerilog.
typedef struct{
    logic [7:0] data_in;
    bit   [1:0] op_sel;
} stim_t;

module multi_op_processor_tb;

    logic [7:0] data_in;
    bit   [1:0] op_sel;
    logic [7:0] data_out_dut;

    stim_t      stimulus_array[];
    logic [7:0] Queue_Output[$];
    logic [7:0] Expected_Values[int];

    function void configure_stim_storage (int size);
        stimulus_array = new[size];
        Queue_Output.delete();
        Expected_Values.delete();
    endfunction

    function automatic void generate_stimulus (ref stim_t stimulus_array[]);
            foreach(stimulus_array[i]) begin
                stimulus_array[i].data_in = $urandom();
                stimulus_array[i].op_sel  = $urandom_range(0,3);
            end
    endfunction

    task drive_stim_and_collect ();
        foreach(stimulus_array[i]) begin
            data_in = stimulus_array[i].data_in;
            op_sel  = stimulus_array[i].op_sel;
            #1ns;
            collect_output_data();
        end
    endtask

    task automatic golden_model();
        foreach(stimulus_array[i]) begin
            case (stimulus_array[i].op_sel)
                2'b00: Expected_Values[i] = stimulus_array[i].data_in + 1;      _
                2'b01: Expected_Values[i] = stimulus_array[i].data_in - 1;      _
                2'b10: Expected_Values[i] = ~stimulus_array[i].data_in;         _
                2'b11: Expected_Values[i] = stimulus_array[i].data_in << 1;     _
                default: Expected_Values[i] = stimulus_array[i].data_in;
            endcase
        end
    endtask

    function void collect_output_data ();
            Queue_Output.push_back(data_out_dut);
    endfunction

    task check_results (logic [7:0] Expected_Values[int], logic [7:0] Queue_Output[$]);
        foreach(Expected_Values[i]) begin
            if (Expected_Values[i] === Queue_Output[i]) begin
                $display("Mismatch at index %0d: Expected %0d, Got %0d", i, Expected_Values[i], Queue_Output[i]);
            end else begin
                $display("Match at index %0d: Value %0d", i, Expected_Values[i]);
            end
        end
    endtask

    function automatic void reconfigure_stim (ref stim_t stimulus_array[]);
        stimulus_array.shuffle();
    endfunction

    multi_op_processor DUT(
        .data_in(data_in),
        .op_sel(op_sel),
        .data_out(data_out_dut)
    );
```

codesnap.dev

Figure 1 Declaration of Functions and Tasks

```
    initial begin

        // Requirement: Configure size with 100 locations, generate, drive, and check results
        $display("Executing Requirement: 100 locations test");
        configure_stim_storage(100);
        generate_stimulus(stimulus_array);
        golden_model();
        drive_stim_and_collect();
        check_results(Expected_Values,Queue_Output);

        #10ns;

        // Requirement: Repeat the process with 200 locations
        $display("Executing Requirement: 200 locations test");
        configure_stim_storage(200);
        generate_stimulus(stimulus_array);
        golden_model();
        drive_stim_and_collect();
        check_results(Expected_Values,Queue_Output);

        #10ns;

        // Requirement: Shuffle stimulus array, clean arrays, recalculate, drive, and check results
        $display("Executing Requirement: Shuffle and re-test");
        reconfigure_stim(stimulus_array);
        Queue_Output.delete();
        Expected_Values.delete();
        golden_model();
        drive_stim_and_collect();
        check_results(Expected_Values,Queue_Output);

        #10ns;
        $finish;
    end

endmodule
```

codesnap.dev

Figure 2 Initial Block (Main Code)

```
vlib work
vlog multi_op_processor_tb.sv multi_op_processor.sv
vsim -voptargs=+acc work.multi_op_processor_tb
add wave *
run -all
#quit -sim
```

codesnap.dev

Figure 3 Do file code for automating the process.

2. Snippet for the waveform shows the values for created variables

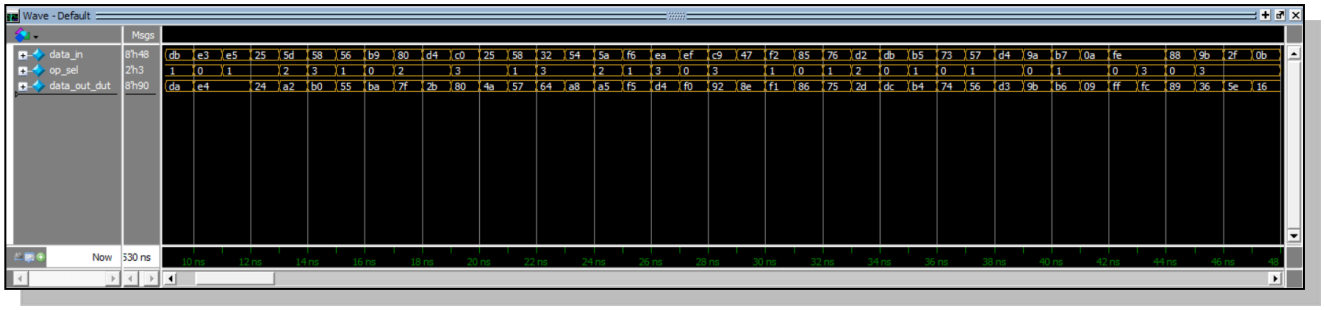

Figure 4 Waveform shows the created variables

3. Snippets for the logs show the all printed values.

```
# Executing Requirement: 100 locations test
# Match at index 0: Value 71
# Match at index 1: Value 153
# Match at index 2: Value 57
# Match at index 3: Value 141
# Match at index 4: Value 152
# Match at index 5: Value 107
# Match at index 6: Value 144
# Match at index 7: Value 54
# Match at index 8: Value 173
# Match at index 9: Value 218
# Match at index 10: Value 228
# Match at index 11: Value 228
# Match at index 12: Value 36
# Match at index 13: Value 162
# Match at index 14: Value 176
# Match at index 15: Value 85
# Match at index 16: Value 186
# Match at index 17: Value 127
# Match at index 18: Value 43
# Match at index 19: Value 128
# Match at index 20: Value 74
# Match at index 21: Value 87
# Match at index 22: Value 100
# Match at index 23: Value 168
# Match at index 24: Value 165
# Match at index 25: Value 245
# Match at index 26: Value 212
# Match at index 27: Value 240
# Match at index 28: Value 146
# Match at index 29: Value 142
# Match at index 30: Value 241
# Match at index 31: Value 134
# Match at index 32: Value 117
# Match at index 33: Value 45
# Match at index 34: Value 220
# Match at index 35: Value 180
# Match at index 36: Value 116
```

Figure 5 LOG (Requirement 1 test 100) {1}

```
# Match at index 36: Value 116
# Match at index 37: Value 86
# Match at index 38: Value 211
# Match at index 39: Value 155
# Match at index 40: Value 182
# Match at index 41: Value 9
# Match at index 42: Value 255
# Match at index 43: Value 252
# Match at index 44: Value 137
# Match at index 45: Value 54
# Match at index 46: Value 94
# Match at index 47: Value 22
# Match at index 48: Value 255
# Match at index 49: Value 105
# Match at index 50: Value 5
# Match at index 51: Value 94
# Match at index 52: Value 68
# Match at index 53: Value 109
# Match at index 54: Value 170
# Match at index 55: Value 202
# Match at index 56: Value 196
# Match at index 57: Value 160
# Match at index 58: Value 143
# Match at index 59: Value 22
# Match at index 60: Value 136
# Match at index 61: Value 167
# Match at index 62: Value 107
# Match at index 63: Value 84
# Match at index 64: Value 158
# Match at index 65: Value 169
# Match at index 66: Value 211
# Match at index 67: Value 211
# Match at index 68: Value 95
# Match at index 69: Value 7
# Match at index 70: Value 129
# Match at index 71: Value 80
# Match at index 72: Value 176
# Match at index 73: Value 129
```

Figure 6 LOG (Requirement 1 test 100) {2}

```
# Executing Requirement: 200 locations test
# Match at index 0: Value 10
# Match at index 1: Value 110
# Match at index 2: Value 219
# Match at index 3: Value 233
# Match at index 4: Value 83
# Match at index 5: Value 184
# Match at index 6: Value 172
# Match at index 7: Value 225
# Match at index 8: Value 10
# Match at index 9: Value 2
# Match at index 10: Value 129
# Match at index 11: Value 18
# Match at index 12: Value 36
# Match at index 13: Value 139
# Match at index 14: Value 225
# Match at index 15: Value 80
# Match at index 16: Value 94
# Match at index 17: Value 180
# Match at index 18: Value 30
# Match at index 19: Value 88
# Match at index 20: Value 111
# Match at index 21: Value 151
# Match at index 22: Value 255
# Match at index 23: Value 135
# Match at index 24: Value 90
# Match at index 25: Value 207
# Match at index 26: Value 154
# Match at index 27: Value 121
# Match at index 28: Value 130
# Match at index 29: Value 36
# Match at index 30: Value 22
# Match at index 31: Value 244
# Match at index 32: Value 30
# Match at index 33: Value 82
# Match at index 34: Value 212
# Match at index 35: Value 65
# Match at index 36: Value 61
```

Figure 7 LOG (Requirement 2 test 200) {1}

```
# Match at index 37: Value 251
# Match at index 38: Value 99
# Match at index 39: Value 174
# Match at index 40: Value 66
# Match at index 41: Value 130
# Match at index 42: Value 18
# Match at index 43: Value 212
# Match at index 44: Value 113
# Match at index 45: Value 176
# Match at index 46: Value 168
# Match at index 47: Value 20
# Match at index 48: Value 182
# Match at index 49: Value 30
# Match at index 50: Value 33
# Match at index 51: Value 139
# Match at index 52: Value 230
# Match at index 53: Value 114
# Match at index 54: Value 157
# Match at index 55: Value 76
# Match at index 56: Value 45
# Match at index 57: Value 160
# Match at index 58: Value 241
# Match at index 59: Value 162
# Match at index 60: Value 245
# Match at index 61: Value 144
# Match at index 62: Value 239
# Match at index 63: Value 55
# Match at index 64: Value 36
# Match at index 65: Value 232
# Match at index 66: Value 138
# Match at index 67: Value 42
# Match at index 68: Value 221
# Match at index 69: Value 112
# Match at index 70: Value 195
# Match at index 71: Value 146
# Match at index 72: Value 225
# Match at index 73: Value 99
# Match at index 74: Value 168
```

Figure 8 LOG (Requirement 2 test 200) {2}

```
# Executing Requirement: Shuffle and re-test
# Match at index 0: Value 17
# Match at index 1: Value 22
# Match at index 2: Value 100
# Match at index 3: Value 225
# Match at index 4: Value 184
# Match at index 5: Value 109
# Match at index 6: Value 83
# Match at index 7: Value 148
# Match at index 8: Value 61
# Match at index 9: Value 166
# Match at index 10: Value 135
# Match at index 11: Value 121
# Match at index 12: Value 42
# Match at index 13: Value 50
# Match at index 14: Value 76
# Match at index 15: Value 65
# Match at index 16: Value 184
# Match at index 17: Value 114
# Match at index 18: Value 139
# Match at index 19: Value 86
# Match at index 20: Value 225
# Match at index 21: Value 117
# Match at index 22: Value 57
# Match at index 23: Value 224
# Match at index 24: Value 186
# Match at index 25: Value 16
# Match at index 26: Value 152
# Match at index 27: Value 252
# Match at index 28: Value 66
# Match at index 29: Value 186
# Match at index 30: Value 234
# Match at index 31: Value 5
# Match at index 32: Value 203
# Match at index 33: Value 137
# Match at index 34: Value 69
# Match at index 35: Value 148
# Match at index 36: Value 255
```

Figure 9 LOG (Requirement 3 Shuffle) {1}

```
# Match at index 37: Value 31
# Match at index 38: Value 157
# Match at index 39: Value 157
# Match at index 40: Value 87
# Match at index 41: Value 232
# Match at index 42: Value 221
# Match at index 43: Value 162
# Match at index 44: Value 160
# Match at index 45: Value 147
# Match at index 46: Value 10
# Match at index 47: Value 201
# Match at index 48: Value 152
# Match at index 49: Value 102
# Match at index 50: Value 39
# Match at index 51: Value 116
# Match at index 52: Value 185
# Match at index 53: Value 132
# Match at index 54: Value 33
# Match at index 55: Value 239
# Match at index 56: Value 36
# Match at index 57: Value 111
# Match at index 58: Value 180
# Match at index 59: Value 180
# Match at index 60: Value 82
# Match at index 61: Value 192
# Match at index 62: Value 254
# Match at index 63: Value 82
# Match at index 64: Value 189
# Match at index 65: Value 231
# Match at index 66: Value 7
# Match at index 67: Value 144
# Match at index 68: Value 28
# Match at index 69: Value 184
# Match at index 70: Value 233
# Match at index 71: Value 219
# Match at index 72: Value 197
# Match at index 73: Value 224
# Match at index 74: Value 121
```

Figure 10 LOG (Requirement 3 Shuffle) {2}